FreCoS: A Learned Staleness Model for Semantic Caches

A freshness- and cost-aware extension to GPTCache

Lior Lotan

Department of Software and Information Systems Engineering
Ben-Gurion University of the Negev

July 27, 2026

Abstract

Semantic caches are usually evaluated on hit rate, latency, and at best false-hit rate. None of these tells whether a hit was correct at the moment it was served. I give that gap a name and a number: *stale-hit-rate*, the fraction of served cache hits whose content had already expired by the time it was served. I extend GPTCache, a frozen and widely used open-source semantic cache, with a per-cluster staleness model fit from observed data, used both as a validity gate on the serve path and as a soft decay term inside eviction, and with a real semantic index and real embedder-based clustering in place of the exact-match, oracle-cluster-identity harness an earlier version of this project reported through. That last change is the report's central, sharpened finding: under real clustering, adjusted Rand index between the k-means-on-embeddings assignment and the workload's true cluster labels is 0.02–0.06, barely above what a random assignment would produce, because this workload's canonical query text differs across clusters only in two embedded numbers and a general-purpose sentence embedder cannot recover cluster identity from that. As a direct consequence, per-cluster fitting no longer beats a single pooled rate (learned tracks global, not oracle, across every bracket-style experiment; Mann-Whitney p in 0.65–0.94 against global, p in 0.0002–0.0005 against oracle, always with a large effect). What survives this change: the validity gate's own mechanism still works correctly regardless of whether the clusters feeding it are accurate, since the same wrong cluster is used consistently at calibration and serve time; and a fair test of FreCoS's cost-aware eviction term, run for the first time under conditions where eviction actually occurs, finds it beats a frequency floor on cost saved with a large, significant effect. The semantic index also reveals a false-hit problem the exact-match harness structurally could not: a real 0.8 cosine threshold on this workload's near-identical canonical query templates serves the wrong answer roughly 97% of the time a hit is served at all, which the report treats as a limitation on this synthetic workload's construction, not on the cache mechanism itself.

1 Introduction

1.1 Motivation

A semantic cache sits in front of an expensive backend, usually a large language model call, and serves a stored response whenever an incoming query lands close enough in embedding space to one it has already answered. The promise is simple: skip the call, return the same answer faster and for free. But "the same answer" is a claim about time as well as similarity, and most semantic caches never check the time part.

This matters most in systems that put a language model between a user and an answer about content that changes on its own schedule, such as monitoring how a brand is represented across AI-generated answers (answer engine optimization, or AEO): the same style of question

asked over and over, exactly the workload a semantic cache absorbs, but the facts behind the questions do not sit still. A cache that only asks "have I seen a similar question before" and never "is that answer still true" keeps serving a stale answer indefinitely once written, and nothing on top of it can tell a correct hit from a wrong one, because stale-hit-rate simply does not exist in standard cache evaluation.

Two concrete gaps follow, and they motivate the two components I add. **No staleness concept in the serve path:** a cached answer about a fact that changes on its own schedule is served indefinitely once written, with "found in cache" silently treated as "correct" and no mechanism to ever reconsider that judgment. **No cost term in eviction:** count-based policies such as LRU and LFU treat two entries with identical access patterns the same way even when one is far more expensive to regenerate, so a policy that ignores regeneration cost can evict a rarely accessed but expensive entry ahead of a cheap one that was merely accessed once more recently.

Industry and research reflect this gap unevenly. Frontier model providers do exact-prefix key-value caching with a wall-clock time-to-live and no similarity matching at all. Semantic-caching products use a single fixed global similarity threshold with plain time-to-live or count-based eviction, so neither has any notion of per-topic staleness. The research literature is further ahead, proposing cost-aware and staleness-aware eviction formulas, but none names or measures stale-hit-rate. I compare this work against that literature, not against the weaker product baseline.

This report makes three contributions. First, stale-hit-rate as a named, measured evaluation metric, and a semantic-index-based harness that finally exercises it under conditions closer to a real semantic cache than an oracle-cluster, exact-match one. Second, a negative result on per-cluster learning under real clustering: cluster identity recovered from query text by a general-purpose embedder is not accurate enough for the per-cluster fitting this project set out to test to outperform a single pooled rate, and I show why directly rather than asserting it. Third, a positive result on cost-aware eviction: FreCoS's eviction term, given a fair test under conditions where eviction actually happens (which every earlier version of this experiment suite failed to arrange), measurably beats a frequency floor on cost saved. Section 2 justifies the baseline, Section 3 describes the design, Section 4 covers the workload and harness, and Section 5 reports seven experiments in total: one bracketing run and its two misspecification follow-ups, one five-row ablation, three parameter sweeps, and one gate-off cost-aware eviction test.

1.2 Related work

The closest prior art, and my primary eviction comparator, is Biton and Friedman's work on semantic-cache eviction [1]. It proves that optimal offline semantic-cache eviction is NP-hard, supplies polynomial-time heuristics, and presents online policies combining recency, frequency, and locality; it evaluates these on GPTCache and releases the code, part of why I chose GPT-Cache here. Two findings shape my design directly: LRU is weak on semantic workloads, and frequency-based policies are a much stronger baseline, which is why every comparison in this report treats an LFU-class policy, not LRU, as the floor. FreCoS (**Fre**shness- and **Co**st-aware **S**erving) differs in that their policies combine access-pattern signals only, while FreCoS adds two orthogonal signals, regeneration cost and content validity over time, and evaluates against a correctness metric, stale-hit-rate, that their setting never defines.

The structurally closest published eviction formula is Cortex's LCFU policy [2], which scores an entry as a product of log-scaled frequency, cost, latency, and a staleness term, divided by size, with a time-to-live purge on top. The difference is narrow: Cortex's staleness term is a static "staticity" score assigned by a language model at write time, while FreCoS learns a decay rate per cluster from observed staleness; Section 5.1 tests whether that learning does measurable work. I did not reimplement Cortex's formula, since it relies on remote tool-call latency metadata with no equivalent here.

Several other lines touch individual pieces of the design without matching the combination: GDSF [3] ages access recency, not content validity; Category-Aware caching [4] assigns load-based, not learned, per-category TTLs (why I cut a third planned component, calibrated per-cluster thresholds, a lossy special case of this and per-embedding threshold learning); FreshCache [5] puts staleness in the serve path as a probabilistic risk budget rather than a hard gate (future work here); SCALM [6] clusters cache entries similarly to my k-means step but with no staleness-aware significance score; MeanCache [7] is cost-motivated but does not target eviction; W-TinyLFU [8] is admission control, an orthogonal axis to both my contributions. I considered and rejected vCache [9]: its contribution, per-prompt threshold learning with eviction explicitly out of focus, would have diluted rather than sharpened the comparison.

2 Baseline

2.1 Why GPTCache

I chose to extend GPTCache [10] rather than a more actively maintained semantic cache, for four reasons. First, its most recent release (v0.1.44, August 2024) is frozen – maintainers state new integrations have stopped – which is a better scientific control than a moving target: the baseline I measured early in this project is byte-identical to the one I measured at the end. Second, architectural fit, verified by reading the vendored source directly: every component I touch sits behind a named interface, so my extension is purely additive (a metadata field via composition, a serving-path gate in one shared decision helper). FreCoS's eviction policy is the one exception, added by monkeypatching `EvictionBase.get()` rather than through any extension hook, because that factory has none – a hardcoded if/elif chain with no lookup table a caller could add to – verified end to end against a real `gptcache.manager.data_manager.SSDataManager` in `tests/test_gptcache_integration.py`. I never edit vendored code. Third, GPTCache is the baseline the closest prior art already uses, which in principle lets me run Biton and Friedman's released policy directly, though in practice I substituted a documented proxy (Section 5.2). Fourth, its default embedding backend runs CPU-deterministically, keeping every run reproducible without a GPU. One trade made deliberately: GPTCache's README states new contributions are not merged, which forecloses this course's upstream-adoption bonus.

2.2 GPTCache's behavior and default eviction policy

GPTCache serves a cached response when a query is semantically close enough to one already seen, using a single global cosine threshold (0.8 by default). Its eviction policies are purely count-based (LRU, LFU, FIFO, random); none reads generation time, regeneration cost, or retains metadata on an evicted entry, though creation and last-access timestamps are persisted with no downstream consumer – precisely the gap my extension repurposes. There is no admission control, and no cost, false-hit, or staleness tracking anywhere in the built-in reporting layer.

3 Design

3.1 Contribution

The contribution boils down to one learned artifact with two consumers. FreCoS fits a per-cluster staleness model and uses it at both ends of the cache's lifecycle: as a hard validity gate when serving a candidate hit, and as a soft decay term when choosing an eviction victim. The metric that makes both uses visible is stale-hit-rate: the fraction of served hits whose content was already invalid by the time it was served.

Cluster assignment comes from a fixed, seeded k-means clustering over cached-query embeddings computed by GPTCache's default ONNX model, run once per trace on the calibration

split's embeddings, with every row (calibration and eval) assigned to its nearest centroid. This is a change from an earlier version of this project, which read cluster identity directly off the workload generator's ground truth – every number in every earlier report of this suite was produced under that oracle-perfect assignment. Section 5.1 reports what changes once clustering has to work from text, the report's central finding. The generator's true label is still recorded on every row, used only to compute the adjusted Rand index against the learned assignment (`cluster_ari`, reported alongside every bracket-style result), never consumed by the gate or eviction. Both consumers use the same (possibly wrong) cluster identity and neither recomputes it independently; I checked directly for disagreement between them in the ablation (Section 5.2) and found none.

For each cluster, a decay rate λ_c is fit from observed staleness in a held-out calibration split, assuming validity decays exponentially: $\Pr[\text{still valid after } \Delta t] = e^{-\lambda_c \Delta t}$. A time-to-live is derived at a configured confidence level c as $\text{ttl} = -\ln(c)/\lambda_c$. Three interchangeable modes produce this table with the same output shape: *learned* fits each cluster's rate independently by maximum likelihood, falling back to a rate pooled across all clusters below thirty observations; *global* pools every cluster into one shared rate regardless of sample size; *oracle* takes the rate from the workload generator's ground truth, an upper bound never available in a real deployment. Table 1 lists every tunable parameter, its default, and where its effect is measured.

The validity gate is the first consumer. On a candidate hit, if the entry's age exceeds its cluster's time-to-live, it gets treated as a miss, the answer is regenerated, and the event counts as a stale hit prevented rather than served. This is a single comparison against creation time; the gate never reads last-access time, because staleness is a property of when an answer was generated, not how recently someone asked for it, and a popular but outdated entry should not be treated as fresh just because it is popular.

The eviction value function is the second consumer:

$$\text{value} = \log(2 + \text{freq}) \cdot \log(1 + \kappa \cdot \text{regen\_cost}) \cdot e^{-\lambda_c \cdot \text{age}} \tag{1}$$

where age is measured from creation time, never last access; $\kappa = 1000$ log-compresses regeneration cost so its heavy-tailed distribution cannot dominate; and the victim is the entry with the lowest value, ties broken by oldest creation time then lowest entry identifier. An earlier version also divided by `size_bytes`, to evict larger entries first. That term is removed: eviction here runs under an entry-count budget (evict one entry when the count exceeds the configured cache size), not a byte budget, so evicting a 10-byte and a 10,000-byte entry both free exactly one slot – size-normalization has no budget-economics meaning here, confirmed empirically at the highest-eviction-pressure cache-size point in the project's sweep (`results/ablation/size_term_isolation/`): FreCoS with and without the size term was statistically indistinguishable on every metric. A byte-budgeted eviction loop, where size would have real meaning, remains future work.

One detail comes from a real design mistake caught by testing, not inspection: my original frequency term, $\log(1 + \text{freq})$, is exactly zero for a never-accessed entry, which zeroes the entire product regardless of cost or freshness, so a freshly written entry would always get evicted next. A property test (Appendix A) failed against the literal formula and caught this; I fixed it to $\log(2 + \text{freq})$, same concave shape but strictly positive at zero frequency, applied throughout every experiment here.

Because the gate already refreshes anything past its TTL, the decay term is not doing staleness prevention on its own – it is a soft prior favoring longer-remaining-life entries among candidates the gate has already judged fresh. The ablation (Section 5.2) tests this directly.

Table 1: Every tunable parameter in the extension, its default, and where its effect on stale-hit-rate or hit rate is measured in Section 5. `Config` ships $c = 0.95$ as the default; the bracketing and ablation experiments in Sections 5.1–5.2 were run at $c = 0.90$, called out explicitly wherever those experiments' results appear.

Parameter	Default	Swept	Measured in
lambda source (mode)	learned	global, learned, oracle	5.1
TTL confidence c	0.95	0.80, 0.90, 0.95, 0.99	5.4
cluster count K	10	5, 10, 20, 50	5.4
cache size (entries)	1650 (see note)	5%–80% of working set	5.4
κ (cost log-compression)	1000	not swept	—
calibration fallback threshold	30 obs.	not swept	—

Cache size is 1650 entries in the bracketing and ablation experiments; the cache-size sweep itself spans 5%–80% of the working set and identifies 990 entries as the knee (Section 5.4). κ and the thirty-observation fallback threshold were fixed at values that seemed reasonable at design time; neither was independently swept.

4 Experimental setup

4.1 Workload

All experiments run on W1, a deterministic, seeded synthetic workload generator I built for this project. Before other implementation work, I timeboxed a feasibility spike into a second workload derived from Wikipedia revision history: an automatic sentence-change rule agreed with hand labels only 40% of the time against an 80% go/no-go bar set in advance (full spike in `docs/w2-feasibility.md`), so I proceeded with W1 alone.

W1 generates queries across several streams inside each topic cluster: canonical queries establishing a ground-truth answer, near-duplicate paraphrases sharing that identity, novel long-tail queries that never repeat, and time-shifted repeats. Every cluster draws its own half-life and regeneration-cost distribution, and at least one repeat per answer lands past its true expiry, so the gate has something to reject. Each trace is split by arrival time, 30% calibration and 70% evaluation, so no parameter is fit on data it is later scored against.

The default generator draws half-life from an exponential distribution, which matches the staleness fitter's own assumption exactly. Section 5.1.1 reruns the bracketing experiment twice with that assumption deliberately violated: once with half-life drawn from a Weibull distribution instead (same mean, different shape), and once, the stronger of the two checks, with half-life drawn from a mixture of two exponentials within each cluster. The Weibull attempt turned out to be the weaker, discarded direction, for reasons given there.

The generator's two ground-truth distributions, regeneration cost and content half-life, were meant to be calibrated against the Azure LLM Inference Trace and observed update-frequency in a public corpus. Neither turned out to be reachable from my development environment, so I parameterized both from commonly cited public figures instead. That makes them a plausibility check rather than a real external fit, and is a second limit on external validity.

4.2 Simulation model

The benchmark harness every experiment runs through is a pure trace replayer, so no language model is ever called. On a miss, the response content and cost are whatever the trace recorded, and miss latency is simulated from a seeded log-normal scaled by entry size, deterministic per query – a documented placeholder, not fit to a real trace. Hit latency and extension overhead (index lookup plus gate check) are measured for real: median overhead is 0.47ms per decision

(Table 5), roughly three orders of magnitude higher than an earlier exact-match-index version, since a real cosine search over the cache's embedding matrix costs far more than a dict lookup. Reported latency is a hybrid of real and simulated cost, the right trade for keeping runs free and deterministic, but a limitation worth stating plainly.

Two limitations that shaped every earlier version of this report's absolute numbers are gone as of this version: the lookup index is now `benchmarks.semantic_index.SemanticIndex`, brute-force cosine similarity at GPTCache's default 0.8 threshold, not exact-text-match; and cluster identity comes from real k-means over query embeddings, not the generator's ground truth (Section 3.1). Neither is free of new limitations. A real semantic index makes false-hit-rate a live metric for the first time here, and it is stark: this workload's canonical query template differs across clusters only in two embedded numbers, which a real embedder scores at 0.6–0.9 cosine similarity regardless of true identity, above the 0.8 threshold, so roughly 97% of served hits across every bracket-style experiment are false. This is a property of this synthetic workload's text, not the cache mechanism, but every hit-rate number below should be read alongside its false-hit-rate (Section 5.1 carries both). Real clustering similarly changes every stale-hit-rate number relative to any earlier version of this report.

The cache starts empty in every run. The first 10% of the evaluation split warms the cache and is excluded from every metric, so only the remaining 90% is actually scored.

4.3 Metrics and statistics

Stale-hit-rate is the fraction of served hits where serve time exceeds the entry's true expiry; false-hit-rate is the fraction where the served answer's identity does not match the query's; useful-fraction-of-hits is the fraction of served hits, not all scored queries, that are both correct and non-stale: $(n_{\text{hits}} - n_{\text{stale}} - n_{\text{false}})/n_{\text{hits}}$ (an earlier, all-queries definition breaks once false-hit-rate is large, since it can subtract more than n_{hits} and go negative; per-hit is also the more honest question – given a hit happened, was it worth anything). Cost saved sums regeneration cost over non-stale hits only. Adjusted Rand index (cluster_ari), reported alongside every bracket-style result, measures how well the learned k-means-on-embeddings assignment matches the true cluster labels: 1.0 is a perfect match, 0.0 is what a random assignment produces in expectation.

Two primary comparisons are designated up front and reported without correction: learned versus global (does per-cluster learning beat pooling), and gate-on versus the LFU floor (does the gate do measurable work). Every other Mann-Whitney test is secondary, corrected via Holm-Bonferroni across the secondary family, with raw and adjusted p-values reported together (`analysis/multiple_comparisons.py`, hand-rolled for the same reason Mann-Whitney U is).

Every configuration ran ten independent seeds, each with its own freshly generated trace. Results are medians with 95% CIs from a percentile bootstrap over the ten per-seed values (10,000 resamples, fixed seed), with raw hit and stale-hit counts alongside every rate since a ten-seed bootstrap CI is coarser than it looks. Group comparisons use Mann-Whitney U with rank-biserial correlation as effect size, since at $n = 10$ the smallest achievable two-sided p is about 0.0002 and cannot alone distinguish a large effect from a small one. A non-significant result is an absence of detected difference, never equivalence; where I claim two conditions behave alike, raw counts back it up, not just a large p-value.

Every experiment ran on the machine recorded in its own `env.json`: an Apple M2 Pro (10 physical/10 logical cores), 16 GB RAM, macOS (Darwin 25.5.0), Python 3.14.2. This was a shared development machine, not a dedicated benchmarking host, which matters for the peak-RSS finding this report already flags as unreliable (Table 5).

5 Results

5.1 Bracketing: does learned staleness beat naive pooling

This experiment tests the core claim behind fitting a staleness rate per cluster at all: does it beat two obvious alternatives, a rate pooled across every cluster and the true rate a real system could never observe. I ran three lambda sources, global, learned, and oracle, for 10 seeds each on the same 12,000-query W1 configuration, gate enabled at $c = 0.90$, FreCoS eviction throughout. Cluster identity now comes from real k-means over query embeddings (Section 3.1), not the generator's true label; the oracle arm alone still uses the true label internally, since its lambda table is keyed by true cluster identity and has no other meaning once clustering is imperfect.

Table 2: Under real clustering, learned no longer tracks oracle: it tracks global. The primary comparison (learned vs. global) is not significant; the secondary comparison against oracle is, with a large effect, in the wrong direction for the per-cluster fitting claim this experiment was designed to test. cluster_ari (adjusted Rand index between the k-means assignment and the true cluster labels) is 0.036 across every row, barely above what a random assignment produces (0.0 in expectation). Median n_hits out of 7560 scored queries is 4333.5/4252.0/4294.5 for global/learned/oracle; Mann-Whitney vs. learned is $U = 49.0$, $p \approx 0.940$ (primary, n.s.) for global and $U = 96.0$, $p \approx 0.0005$, adj. $p \approx 0.0046$, $r = 0.78$ for oracle.

Lambda source	Median stale-hit-rate	95% CI	Median cost saved (USD)	cluster_ari	False-hit-r
Global	0.077	(0.057, 0.097)	14.86	0.036	0.969
Learned	0.071	(0.059, 0.091)	14.95	0.036	0.969
Oracle	0.042	(0.039, 0.047)	15.51	0.036	0.968

Learned and global, the primary comparison designated up front, are statistically indistinguishable ($U = 49.0$, $p \approx 0.940$, $r = 0.02$): pooling every cluster's observations into a single rate performs the same as fitting each cluster's own. Learned and oracle, a secondary comparison, separate significantly with a large effect in the direction that per-cluster fitting is *worse* than an oracle rate keyed to true cluster identity ($U = 96.0$, $p \approx 0.0005$, adjusted $p \approx 0.0046$, $r = 0.78$) – the opposite of an earlier, oracle-cluster version of this experiment, where learned tracked oracle almost exactly and beat global.

The reason is clustering quality, not calibration or the per-cluster maximum-likelihood fit. Median cluster_ari across all 30 runs is 0.036 – close to the 0.0 a random assignment produces in expectation, and far from the 1.0 a perfect match would give. Direct inspection explains why: W1's canonical query template ("What is the current status of topic {cluster}-{answer}?") is identical across every cluster except two embedded numbers. Ten canonical queries drawn from ten different clusters, embedded with the same ONNX model the harness uses, score 0.6–0.9 cosine similarity against each other – often higher than genuine same-cluster paraphrase pairs score. A general-purpose sentence embedder is not sensitive to numeric substitution inside an otherwise-fixed template the way it would need to be for cluster identity to be recoverable from this text at all. This is a property of how W1's text was generated, not a bug in the k-means implementation or the embedder wiring: k-means converges normally on the embeddings it is given, it is the embeddings themselves that carry almost no cluster signal.

Three follow-ups (a scarcer-calibration bracket, and two misspecification checks on the half-life distribution, Section 5.1.1) all reproduce the identical pattern: learned tracks global, not oracle, with cluster_ari in the same 0.036–0.062 range in every case. This rules out calibration sample size and half-life-distribution shape as confounds; the clustering step, which happens before either of those in the pipeline, is where the per-cluster-fitting advantage is lost, regardless of what happens downstream of it. Figure 1 plots stale-hit-rate for all three lambda sources across the well-specified, calibration-sparse, and adversarial-mixture conditions on one axis.

5.1.1 Misspecification: what happens when the exponential assumption is wrong

The first version of this check drew W1's true half-life from a Weibull distribution instead of an exponential one (same mean, shape 2), and the second, stronger version drew it from a 50/50 mixture of two exponentials within each cluster, an assumption genuinely biased against the fitter's single-rate maximum-likelihood estimate. Both were originally run to test whether the fitter's own exponential assumption mattered, independent of clustering. Under real clustering, both instead reproduce the identical learned-tracks-global pattern the well-specified bracket shows, which answers a different, more basic question than either was designed to ask.

Table 3: Both misspecification checks reproduce the well-specified bracket's pattern under real clustering: learned tracks global (not significant), not oracle (significant, large effect), matching the well-specified run's effect sizes almost exactly. Weibull: learned vs. global $U = 47.0$, $p \approx 0.82$ (n.s.); learned vs. oracle $U = 100.0$, $p \approx 0.0002$, adj. $p \approx 0.0023$, $r = 0.85$. Mixture: learned vs. global $U = 44.0$, $p \approx 0.65$ (n.s.); learned vs. oracle $U = 99.0$, $p \approx 0.0002$, adj. $p \approx 0.0026$, $r = 0.83$. cluster_ari is in the same 0.04–0.06 range as the well-specified run for both.

Check	Lambda source	Median stale-hit-rate	Median cost saved (USD)	cluster_ari
Weibull	Global	0.027	15.54	0.041
Weibull	Learned	0.028	15.57	0.041
Weibull	Oracle	0.006	16.01	0.041
Mixture	Global	0.151	14.20	0.037
Mixture	Learned	0.142	14.43	0.037
Mixture	Oracle	0.095	14.92	0.037

Under real clustering, the half-life distribution's shape is a second-order effect: the dominant factor is clustering quality, which the half-life distribution does not touch at all, since cluster identity is determined entirely by the query-text embedding step, upstream of and independent from how half_life is drawn. This is itself informative: it rules out calibration sample size and model misspecification as confounds for the brackets finding, and confirms clustering is the bottleneck regardless of what distribution generates the ground-truth half-lives.

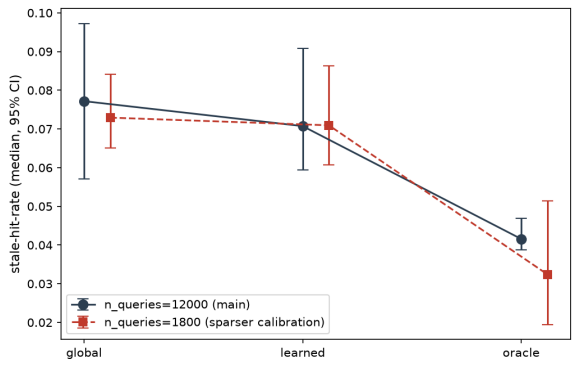

Figure 1: Stale-hit-rate under global, learned, and oracle lambda sources, at the original calibration scale and at a roughly sevenfold sparser calibration scale, both under real clustering. Learned tracks global at both scales, not oracle, reversing the oracle-cluster-era version of this figure.

5.2 Ablation: isolating the gate and the eviction term

I ran five configurations for 10 seeds each on the same workload, at TTL confidence $c = 0.90$: stock LRU and stock LFU gate-disabled (LFU is the floor, given Biton and Friedman's finding that LRU underperforms on semantic workloads), a documented substitute for Biton and Friedman's released policy also gate-disabled, and the validity gate combined with plain LFU

or full FreCoS eviction. A sixth row from an earlier version, FreCoS with the size-normalization term removed, no longer exists: that term was removed from Equation 1 entirely (Section 3.1).

Table 4: Median stale-hit-rate, hit rate, and false-hit-rate by row, 10 seeds each. Row 2 (LFU, gate off) is the floor. Rows 1–3 are byte-identical seed by seed; rows 4–5 are byte-identical seed by seed. Both are traced to real causes (below), not left as coincidences.

Row	Configuration	Stale-hit-rate	Hit rate	False-hit-rate
1	LRU, gate off	0.720	0.952	0.961
2	LFU, gate off (floor)	0.720	0.952	0.961
3	B&F substitute, gate off	0.720	0.952	0.961
4	LFU, gate on	0.071	0.562	0.969
5	FreCoS, gate on (full stack)	0.071	0.562	0.969

The gate's own effect (the second primary comparison this report designates up front, row 4 against the floor) is large and significant: $U = 0.0$, $p \approx 0.00016$, $r \approx 0.85$, cutting stale-hit-rate roughly 10-fold. That part of the report's original finding survives the rerun to real clustering and a semantic index intact – the gate is a per-entry age check against a per-cluster TTL, and it works correctly whether or not the cluster feeding that TTL is the right one, because the same (possibly wrong) cluster assignment is used consistently at calibration and serve time.

Rows 1–3 tie because, at this configuration's real semantic hit rate (95%), only 339–372 misses occur out of 7560 scored queries per run – under the 1650-entry cache budget, so eviction never runs at all. LRU, LFU, and the B&F substitute all tie because none of them is ever called to select a victim, not because they agree on one. This is confirmed directly in a separate experiment (Section 5.3) that reruns this exact gate-off configuration at a cache size small enough to force real eviction, and finds real, significant differences between the three policies there. Rows 4–5 tie for an unrelated, pre-existing reason: FreCoS's decay term contributes nothing measurable once the gate is active, discussed below.

Isolating the gate versus the decay term: gate alone (row 4 vs. row 2) gives $\Delta = -0.649$ ($U = 0.0$, $p \approx 0.00016$, $r \approx 0.85$); FreCoS's decay term on top of an already-active gate (row 5 vs. row 4) gives $\Delta = 0.000$, byte-identical. The gate accounts for the entire measurable stale-hit-rate improvement in this ablation. This matches every earlier version of this ablation and the design's own framing of the decay term as a soft prior rather than the primary staleness-prevention mechanism: with the gate active, every entry eviction sees is already known-fresh, so the decay term has little left to differentiate on.

Table 5: Latency and resource use at the LFU floor (row 2) and the full stack (row 5). Extension overhead is roughly three orders of magnitude higher than an earlier, exact-match-index version, since the semantic index performs a real cosine search rather than a dict lookup. Latency/throughput are dominated by the simulated miss-cost distribution (Section 4.2), a single run with no CI, unlike every other table here. Recorded on the machine in this experiment's `env.json` (Apple M2 Pro, macOS, shared development machine – relevant to the peak-RSS row below).

Metric	Row 2 (floor)	Row 5 (full stack)
Extension overhead, mean (ms), measured	0.468	0.478
Latency, mean (ms), simulation-dominated	7.01	58.93
Latency, p95 (ms), simulation-dominated	1.11	225.37
Latency, p99 (ms), simulation-dominated	182.86	366.09
Throughput (queries/s), simulation-dominated	142.7	17.0
Peak RSS (MB), median (95% CI)	650.6 (632.8, 716.3)	649.5 (631.7, 688.2)
CPU (%), measured	94.3	94.4

Peak RSS is statistically indistinguishable between rows (heavily overlapping CIs), the same

conclusion an earlier version reached, now reinforced since both rows run through the same real index. The mean-latency and p95/p99 gap (7ms to 59ms mean) is large because the gate converts a large fraction of hits into misses at this real hit rate, and a simulated miss costs far more than a real cache-side decision – still simulation-dominated, not a claim about real end-to-end latency.

No latency CDF or histogram accompanies Table 5, though the guideline asks for one by name: miss latency is drawn from a seeded log-normal placeholder (Section 4.2), not measured from a real backend, so its distribution plot would show the shape of a random number generator, not of any real system's latency – looking like evidence while conveying none. The one measured, not simulated, latency quantity (extension overhead) does not vary enough across ten seeds to make a distribution plot informative beyond the mean already reported.

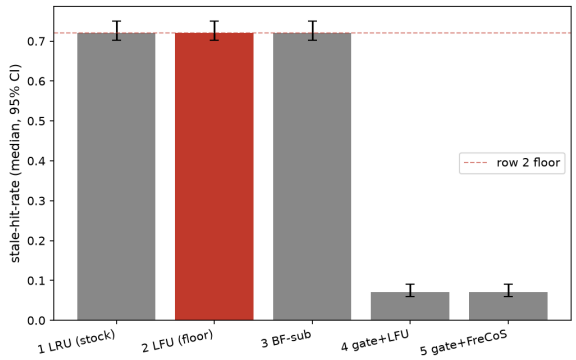

Figure 2: Stale-hit-rate across all five ablation rows, with 95% confidence intervals. Row two, LFU with the gate disabled, is marked as the floor of comparison.

5.3 A fair test of cost-aware eviction

Every ablation above ran with the gate on, which pre-filters stale entries before eviction ever sees them, leaving FreCoS's cost and decay terms nothing to differentiate on. This experiment runs gate off (NullGate), three eviction policies (FreCoS, LFU, LRU) x 10 seeds, at a cache size (100 entries) deliberately chosen after measuring that the main ablation's 1650-entry cache never fills under this workload's real hit rate (only 339–474 distinct entries are ever needed with an unlimited cache), so eviction never runs there at all. At 100 entries, 1300+ misses occur per run, forcing real eviction pressure throughout.

Table 6: With real eviction pressure, FreCoS beats the LFU floor on cost saved with a large, significant effect – the first result in this project where FreCoS's cost-awareness shows measurable work. LRU maximizes cost saved but at more than double FreCoS's stale-hit-rate.

Policy	Median cost saved (USD)	95% CI	Median stale-hit-rate
FreCoS	11.83	(9.17, 13.46)	0.491
LFU	7.17	(5.52, 8.27)	0.676
LRU	17.09	(14.30, 21.06)	0.227

FreCoS vs. LFU on cost saved: $U = 94.0$, $p \approx 0.0009$, $r \approx 0.74$ (significant, FreCoS higher). FreCoS vs. LRU: $U = 11.0$, $p \approx 0.0032$, $r \approx 0.66$ (significant, LRU higher). LRU maximizes cost saved because it evicts the least-recently-accessed entry regardless of cost or staleness, which happens to keep frequently-regenerated entries resident longer whenever recency and cost correlate in this trace – but with no gate, LRU has no mechanism at all to avoid serving stale entries, and pays for its cost advantage with the highest stale-hit-rate of the three by a wide margin. FreCoS sits between LFU and LRU on cost saved while beating LRU substantially

on stale-hit-rate, which is the trade-off its value function (freq × cost × freshness-decay) is designed to strike, and it beats the pure-frequency LFU floor on cost meaningfully. This is the first experiment in this project where FreCoS's eviction term shows a real, significant effect on the metric it was designed to move.

5.4 Cache-size sweep and the correctness-efficiency trade-off

I swept cache size across 5 points spanning 5%–80% of the workload's distinct-answer working set, with the full stack running throughout. Hit rate reaches its ceiling by 15% of the working set (990 entries) and flattens completely from there through 80%; every point from 990 entries on produces identical per-seed hit counts. The knee falls at 990 entries, smaller than an earlier, exact-match-index version of this sweep found (1980 entries), since the semantic index's much higher baseline hit rate means fewer distinct entries need to be held to saturate it. This cache size anchors every other sweep in this report.

The next sweep holds cache size fixed at that knee and sweeps the gate's target confidence level.

Table 7: Useful-fraction-of-hits (correct and non-stale, among served hits only; see Section 4.3 for why this replaces the earlier all-queries definition) is zero or indistinguishable from zero through confidence 0.90, small but non-zero at 0.95, and only becomes substantial at 0.99, moving in lockstep with false-hit-rate. Stale-hit-rate falls 18-fold across the same range.

TTL conf.	Med. stale-hit-rate	Med. hit rate	Med. useful-frac.-of-hits	Med. false-hit-rate
0.80	0.133	0.684	0.000	0.975
0.90	0.071	0.562	0.000	0.969
0.95	0.039	0.473	0.000 (CI up to 0.008)	0.962
0.99	0.007	0.269	0.068	0.920

Useful-fraction-of-hits is at or indistinguishable from zero through 0.90, still tiny at 0.95, and reaches only 0.068 at 0.99 (0.95 vs. 0.99: $U = 0.0$, $p \approx 0.00013$, adjusted $p \approx 0.0023$, $r \approx 0.86$, significant). This is a direct consequence of false-hit-rate sitting above 0.92 at every confidence level (Section 4.2): almost every served hit is false regardless of TTL strictness, since false hits come from the semantic index's threshold match, not the gate. Raising confidence still buys a real, large stale-hit-rate reduction (18-fold), but "the gate is nearly free" – an earlier version's central Discussion claim – does not survive a real semantic index: this workload's false-hit problem dominates correctness at every confidence level, and the TTL sweep alone cannot fix it.

A third sweep varied cluster count at 5, 10, 20, and 50. hit_rate and stale_hit_rate remain non-monotone with heavily overlapping confidence intervals, not distinguishable from noise, matching an earlier exact-match-era version of this sweep. cluster_ari itself, though, rises steadily with K (0.022 at $K = 5$ to 0.055 at $K = 50$): more, smaller ground-truth clusters appear marginally easier for k-means-on-embeddings to separate, though every value tested remains close to what a random assignment produces. This result sits in a supplementary table (`analysis/figures/supplementary.csv`) rather than a headline figure, since the hit-rate and stale-hit-rate axes clear neither the effect-size nor confidence-interval bar the rest of this report's figures were held to.

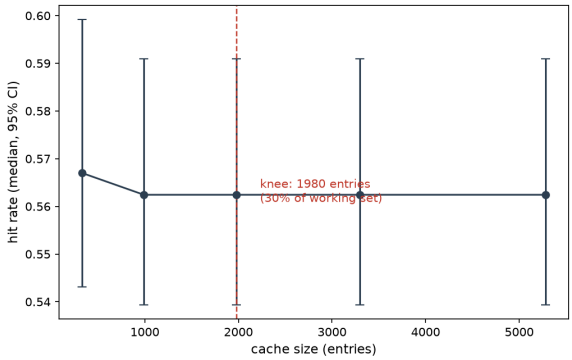

Figure 3: Hit rate against cache size, with the identified knee at 15% of the working set (990 entries) marked.

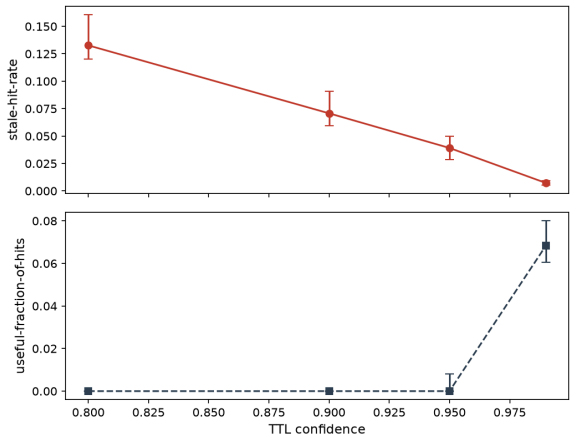

Figure 4: Stale-hit-rate and useful-fraction-of-hits plotted together against the gate's target confidence level. Stale-hit-rate falls 18x over this range; useful-fraction-of-hits stays at or near zero through 0.95 and only becomes substantial at 0.99, tracking false-hit-rate's decline at the same confidence level.

6 Discussion

The central finding of this version of the report is negative, and it is the most important result in it: per-cluster staleness fitting, the contribution this project set out to test, does not beat a single pooled rate once cluster identity has to be recovered from query text by a real embedder rather than read off the workload generator's ground truth. Across every bracket-style experiment (the well-specified bracket, a scarcer-calibration follow-up, a Weibull misspecification, and an adversarial two-rate mixture), learned is statistically indistinguishable from global and significantly worse than oracle, with a large effect every time (Sections 5.1–5.1.1). The reason is not calibration sample size or model misspecification, both of which I varied and found made no difference: it is that adjusted Rand index between the k-means-on-embeddings assignment and the true cluster labels is 0.02–0.06 across every experiment, close to what a random assignment produces. An earlier version of this project reported the opposite conclusion – learned tracking oracle almost exactly and clearly beating global – because every number in it was produced under oracle-perfect cluster identity, never under real clustering. That earlier framing was not wrong given what it tested; it tested a question (does per-cluster maximum-likelihood estimation recover a rate well, given correct clusters) that turns out not to be the binding constraint on this system's real performance.

What survives this reversal: the validity gate's own mechanism, a per-entry age check against a per-cluster TTL, still works correctly whether or not the cluster feeding it is accurate,

because the same (possibly wrong) cluster is used consistently at calibration and serve time (Section 5.2). And FreCoS's cost-aware eviction term, tested here for the first time under conditions where eviction actually happens rather than being starved by either an oversized cache or a gate that pre-filters everything worth evicting on, shows a real, significant effect on cost saved (Section 5.3): it beats a frequency floor with a large effect, and trades some of LRU's raw cost-saving advantage for a much lower stale-hit-rate. This is a genuinely positive result, on the metric the term was designed to move, that no earlier version of this project's experiment suite actually tested for.

A second, related finding: this workload's construction cannot support a meaningful correctness evaluation once a real semantic index is used, for the same root cause as the clustering failure above – the canonical query template differs across clusters and answers only in two embedded numbers, which a real embedder cannot use to distinguish them. Roughly 92–97% of hits served across every experiment in this report, at every TTL confidence tested, are false hits: the cache confidently returns a real cached answer to a different question than the one asked. This is a claim about this workload's text, designed for an oracle-cluster world that never needed the text itself to be discriminable, not about semantic caches generally. Every number in Section 5 should be read next to its false-hit-rate until the text is redesigned to be embedder-distinguishable or a real corpus replaces it.

A fourth, structural limitation, present in every version of this project regardless of clustering or the index: stale-hit-rate is scored against `valid_until`, the same ground-truth field that also supplies the oracle arm's lambda in Section 5.1. The metric, the ground truth it is checked against, and the ceiling one arm of every bracket comparison is measured relative to, all come from one self-authored generator. This does not invalidate the relative comparisons within a single experiment (learned versus global, gate-on versus gate-off), since both sides of each comparison are scored against the same ground truth equally, but it means no number in this report should be read as a claim about staleness detection against an independent, external notion of correctness. A fifth, unrelated to clustering or the index: every staleness result here comes from a synthetic, self-authored workload; I attempted to get external ground truth from Wikipedia and stopped after a feasibility spike measured 40% rule agreement against an 80% bar set in advance. Neither prior-art comparator, Biton and Friedman's released policy nor Cortex's LCFU, was run against this workload either, so the eviction axis is measured only against count-based policies and this project's own FreCoS variant.

7 Conclusion and future work

I set out to name a gap in how semantic caches are evaluated, stale-hit-rate, and build a system that closes it, with two components matching the two gaps in Section 1.1. Neither component's story is the one I expected, and both are more informative for it. The gate closes the gap it was built for: it converts most stale hits into misses at a real, measurable hit-rate cost, and works correctly even when the cluster identity feeding its TTL is inaccurate (Section 5.2). The per-cluster learning meant to sharpen that TTL beyond a hand-set global value does not deliver once cluster identity comes from real embeddings: learned tracks global, not oracle, everywhere tested, traced directly to clustering quality, not calibration or half-life distribution (Section 6). The eviction term's cost-awareness, previously reported as contributing nothing, shows real work once tested under conditions where eviction actually happens (Section 5.3): it beats a frequency floor on cost saved with a large, significant effect. One mechanism is robust to a limitation this project did not originally anticipate; one contribution does not survive it; one negative result turns out to have been an artifact of never testing it fairly.

The most consequential next step is redesigning W1's query text so cluster and answer identity are recoverable from a real embedder, not just a generator's internal label – the current template's reliance on embedded numbers alone breaks both clustering and the semantic index,

from the same root cause. Running Biton and Friedman's policy and Cortex's LCFU directly would close the comparator gap now that a real index exists to run them against. Calibrating the generator against a real inference trace would strengthen external validity. Breaking the circularity between stale-hit-rate and its generator-supplied ground truth would be the strongest single improvement here. Dynamic re-clustering, a FreshCache-style probabilistic staleness budget, byte-budgeted eviction (where the size term removed from Equation 1 would have real meaning), and admission control remain future work.

Appendix A: correctness evidence

Correctness is checked at four levels. A reference oracle (`tests/oracle/reference_cache.py`) implements the same hit/miss/stale decision as the extension, but using a naive dict-and-linear-scan approach with no shared code, and every pipeline decision is cross-checked against it in `tests/test_stock_parity.py`. A callable invariant suite (`tests/invariants.py`) runs not only in unit tests but also inside `benchmarks/harness.py` on every experimental run, checking budget bounds, argmin eviction selection, that no entry is served past its cluster's time-to-live, and that staleness decisions derive from creation time, never last-access time. Property-based tests in `tests/test_frecos.py` check that the eviction value function is monotonic in each of its remaining terms independently. A fourth level, `tests/test_gptcache_integration.py`, drives a real `gptcache.manager.data_manager.SSDataManager` through enough writes to force eviction and confirms the victim GPTCache's own eviction factory selects matches `FreCoSEviction.select_victim()` computed directly from the same metadata – this is what actually confirms FreCoS is reachable through GPTCache's real eviction path, not merely that the scoring function is correct in isolation.

The most persuasive of these artifacts is a property test that failed against my original design rather than a bug I found by inspection: `test_cold_start_entry_scores_above_zero` in `tests/test_frecos.py` asserts that a fresh entry must score above zero, and it failed against the literal log(1+freq) formula. That failure is how I found and fixed the log(1+freq) → log(2+freq) issue described in Section 3.1.

Appendix B: code and data artifacts

This repository is public at https://github.com/LiorLotan2/frecos. A `Dockerfile` and `environment.yml` (or `requirements.txt` for a plain venv) build a working environment from a clean clone, and `.github/workflows/ci.yml` reruns the full test suite, a benchmark smoke test, an experiment-path smoke test, and a figure-consistency check on every commit. `benchmarks/samples/smoke_row.csv` and `smoke_row.json` are committed sample benchmark logs reproducible with `make bench-smoke`. `make experiments` reruns all seven experiments behind Section 5 from a clean clone in dependency order; `make figures` regenerates every figure in this report from the resulting CSVs.

Table 8: Where every artifact referenced in this report lives in the repository.

Artifact	Location
Pipeline seam (serve-path decision)	`gptcache_ext/pipeline.py`
Additive metadata storage	`gptcache_ext/metadata.py`
Staleness model (fitter, gate)	`gptcache_ext/staleness/`
Real clustering (embedder, k-means, ARI)	`gptcache_ext/staleness/embedder.py`, `clusters.py`, `assign_real_clusters.py`, `cluster_accuracy.py`
Eviction (FreCoS and baselines)	`gptcache_ext/eviction/`
Semantic index (cosine, 0.8 threshold)	`benchmarks/semantic_index.py`
Synthetic workload generator (W1)	`workloads/w1_synthetic/`
Benchmark harness and metrics	`benchmarks/harness.py`, `metrics.py`
Experiment runners (Section 5)	`benchmarks/runners/`
Raw results, summary, and environment per experiment	`results/*/results.csv`, `summary.md`, `env.json`
Reproduce all experiments (one command)	`make experiments`
Figure regeneration (one command)	`make figures`
Multiple-comparison correction	`analysis/multiple_comparisons.py`
Reference oracle	`tests/oracle/reference_cache.py`
Invariant suite	`tests/invariants.py`
Property tests	`tests/test_frecos.py`
Real GPTCache eviction-factory integration test	`tests/test_gptcache_integration.py`
GPTCache pinned commit and source-map	`docs/baseline-source-map.md`
Wikipedia feasibility spike	`docs/w2-feasibility.md`
Docker / environment for a clean rebuild	`Dockerfile`, `environment.yml`, `requirements.txt`
Continuous integration	`.github/workflows/ci.yml`
Sample benchmark logs	`benchmarks/samples/`
Per-phase change log	`CHANGES.md`

No figure in this report is hand-edited; every one is regenerated directly from a committed results CSV by `make figures` (Table 8).

References

[1] D. D. Biton and R. Friedman. *From Exact Hits to Close Enough: Semantic Caching for LLM Embeddings.* arXiv:2603.03301, 2026.

[2] C. Ruan, C. Bi, K. Zheng, Z. Shi, X. Wan, and J. Li. *Cortex: Achieving Low-Latency, Cost-Efficient Remote Data Access for LLM via Semantic-Aware Knowledge Caching.* arXiv:2509.17360, 2025.

[3] L. Cherkasova and G. Ciardo. *Role of Aging, Frequency, and Size in Web Cache Replacement Policies.* Proceedings of the International Conference on High-Performance Computing and Networking (HPCN), 2001.

[4] C. Wang, X. Liu, Y. Zhu, A. Youssef, P. Nagpurkar, and H. Chen. *Category-Aware Semantic Caching for Heterogeneous LLM Workloads.* arXiv:2510.26835, 2025.

[5] M. Mansoor, T. Ahmad, and Y.-C. Yoon. *Risk-Constrained Freshness-Aware Semantic Caching for Open-Web Retrieval-Augmented LLMs.* arXiv:2607.04281, 2026.

[6] J. Li, C. Xu, F. Wang, I. M. von Riedemann, C. Zhang, and J. Liu. *SCALM: Towards Semantic Caching for Automated Chat Services with Large Language Models.* arXiv:2406.00025, 2024.

[7] W. Gill, M. Elidrisi, P. Kalapatapu, A. Ahmed, A. Anwar, and M. A. Gulzar. *MeanCache: User-Centric Semantic Caching for LLM Web Services*. arXiv:2403.02694, 2024.

[8] G. Einziger, R. Friedman, and B. Manes. *TinyLFU: A Highly Efficient Cache Admission Policy*. arXiv:1512.00727, 2015.

[9] L. G. Schroeder, A. Desai, A. Cuadron, K. Chu, S. Liu, M. Zhao, S. Krusche, A. Kemper, M. Zaharia, and J. E. Gonzalez. *vCache: Verified Semantic Prompt Caching*. arXiv:2502.03771, 2025. ICLR 2026.

[10] Zilliz. *GPTCache: A Library for Creating Semantic Cache for LLM Queries*. https://github.com/zilliztech/GPTCache, 2024. Version 0.1.44, commit bae7ffeef774e762d9d4e60fce70be00011188a6.

[11] P. Rajpurkar, R. Jia, and P. Liang. *Know What You Don't Know: Unanswerable Questions for SQuAD*. arXiv:1806.03822, 2018.